

An explicit search reduction from Vertex Cover to Set Basis

Reconstruction for expert review
26 September 2026

Abstract

Set Basis asks for a small family of subsets whose unions represent every prescribed set. We give an executable search reduction from Vertex Cover using a published five-set edge gadget. A direct counting argument recovers a vertex cover from every valid basis, including redundant bases; infeasibility is recovered as well. For a graph with m edges, the target has at most $8m$ universe elements and $7m$ family members. Independent solvers checked 112 fixed source cases and seven further boundary cases. The gadget is known; the contribution is an explicit two-algorithm reconstruction and an all-output proof.

1. Introduction

Set Basis formalizes reusable union representations and is equivalent to exact Boolean factorization of a finite incidence matrix. Stockmeyer’s 1975 report established its NP-completeness [1]. The report’s full proof was unavailable for this reconstruction. Jiang and Ravikumar’s later five-set gadget [2] appears explicitly in Björklund and Martens’ Lemmas 4–5 [3]. Their Lemma 4 treats Normal Set Basis, and Lemma 5 connects the constructed instances to ordinary Set Basis.

The fixed task asks for deterministic instance construction and recovery of a Vertex Cover answer from **every** valid Set Basis output. A decision equivalence alone does not specify this recovery algorithm. We reconstruct the published gadget, derive a direct count for ordinary basis witnesses, and implement both maps. Theorem 1 gives the all-output statement. The gadget and core endpoint argument are already published [3]; the original Jiang–Ravikumar and Stockmeyer proofs were not independently inspected.

2. Problems and output contract

A source instance is a finite simple graph $G = (V, E)$ and a nonnegative integer k . Its valid outputs are all vertex covers $T \subseteq V$ with $|T| \leq k$, or the answer NO-SOLUTION exactly when no such cover exists. A target instance is a finite universe U , an explicit family $\mathcal{C}$ of subsets of U , and a nonnegative integer K . A valid basis output is a family $\mathcal{B}$ of at most K subsets such that each $C \in \mathcal{C}$ is the union of some subfamily of $\mathcal{B}$. The empty union is empty. The target returns NO-SOLUTION exactly when no basis exists. Both output sets are nonempty on legal inputs.

The JSON encoding uses vertex labels from 0 to $n - 1$, canonical edge pairs, and integer parameters of unrestricted encoded length. Recovery receives the original source instance and a target output in a fresh process.

3. Construction

Let A be the vertices incident to edges, ordered by source label. Write $q = |A|$ and $m = |E|$, index A by $0, \dots, q - 1$, and identify each active vertex with its index below. Isolated vertices affect no edge and are discarded. For each active vertex i , create universe elements x_i, y_i and family member $c_i = \{x_i, y_i\}$. For each edge $f = \{i, j\}$ with $i < j$, create private elements a_f, b_f, d_f, e_f and add the five members in Equation 1.

$$\begin{aligned} c_f^1 &= \{x_i, a_f, b_f\}, & c_f^2 &= \{y_j, b_f, d_f\}, \\ c_f^3 &= \{y_i, d_f, e_f\}, & c_f^4 &= \{x_j, a_f, e_f\}, \\ c_f^5 &= \{a_f, b_f, d_f, e_f\}. \end{aligned} \tag{1}$$

Figure 1 shows the incidence pattern. Each dot marks inclusion; the vertical rule separates vertex symbols from private edge symbols. The four private symbols have incompatible occurrences among the first four rows.

	x_i	y_i	x_j	y_j		a	b	d	e
c_1	•					•	•		
c_2				•			•	•	
c_3		•						•	•
c_4			•			•			•
c_5						•	•	•	•

Figure 1: Incidence of the five rows for an edge $f = \{i, j\}$. Columns x_i, y_i, x_j, y_j belong to vertex gadgets; a_f, b_f, d_f, e_f occur only in this edge gadget. A dot means membership.

The forward map sets $K = q + 4m + k$. It uses the same rows for every edge and disjoint private elements for different edges. The target has $2q + 4m$ universe elements and $q + 5m$ family members.

3.1. Forward witness

Lemma 1. If G has a cover T with $|T| \leq k$, the target has a basis of size at most K .

Proof. For each $i \in T$, take the singletons $\{x_i\}$ and $\{y_i\}$. For each other active vertex take $\{x_i, y_i\}$. This contributes $q + |T|$ sets. For each edge $f = \{i, j\}$ with $i < j$, take four more sets. If $i \in T$, take $\{a_f, b_f\}$, $\{d_f, e_f\}$, $\{y_j, b_f, d_f\}$ and $\{x_j, a_f, e_f\}$. The first two combine with $\{x_i\}$ and $\{y_i\}$ to form c_f^1 and c_f^3 ; the next two are c_f^2 and c_f^4 ; the first two unite to form c_f^5 . If i is outside T , then $j \in T$. Take $\{a_f, e_f\}$, $\{b_f, d_f\}$, $\{x_i, a_f, b_f\}$ and $\{y_i, d_f, e_f\}$. Now c_f^2, c_f^4 use j 's singletons, c_f^1, c_f^3 are basis sets, and the first two unite to form c_f^5 . Each vertex row is represented, and the total is $q + 4m + |T| \leq K$. $\square$

4. Recovery and correctness

The decoder works with any target basis, including one with unused, empty, or repeated sets. It deletes sets that cannot appear in a row representation. If $\{x_i, y_i\}$ coexists with $\{x_i\}$, it replaces the pair by $\{y_i\}$; it makes the symmetric replacement if $\{y_i\}$ is present. The pair is contained only in the vertex row c_i , so these changes preserve every representation and do not increase the number of sets.

4.1. Local lower bound

Lemma 2. In the normalized basis, let T contain the vertices whose two singletons are present. Every edge gadget needs at least four basis sets containing its private elements. If neither endpoint lies in T , it needs at least five.

Proof. The vertex row c_i forces either its pair alone or both singletons. After normalization there are exactly $q + |T|$ vertex-only basis sets. Every other useful basis set contains private elements of exactly one edge, since it must be a subset of a family member.

Consider one edge f . The four occurrences (c_f^1, a_f) , (c_f^2, b_f) , (c_f^3, d_f) and (c_f^4, e_f) require four distinct edge-local basis sets. For any two occurrences, no basis set can contain both designated elements and be contained in both rows: at least one designated element is absent from the other row. If neither endpoint is split, none of x_i, y_i, x_j, y_j occurs as a singleton basis set. The first four edge rows then each require a distinct edge-local basis set containing their vertex symbol. None of those four sets can represent c_f^5 , which has no vertex symbol, so a fifth edge-local set is needed. $\square$

4.2. All-output reduction

Theorem 1. The construction F and decoder G are deterministic polynomial-time maps. For every legal source input x and every valid target output y for $F(x)$, $G(x, y)$ is a valid source output.

Proof. Let $\mathcal{B}$ be any valid target basis. Normalize it as above and let u count edges with neither endpoint in T . By Lemma 2 and the privacy of edge symbols,

$$|\mathcal{B}| \geq q + |T| + 4m + u. \quad (2)$$

The budget gives $|T| + u \leq k$. The decoder outputs T plus one endpoint of each of those u edges. This meets every edge and uses at most k vertices. It reconstructs all labels from the original graph, so it needs no data retained by the forward process. If y is NO-SOLUTION, Lemma 1 rules out a source cover; the decoder returns the same answer. These cases exhaust the valid target outputs. $\square$

5. Encoding and runtime

At most $2m$ vertices are active. The target has at most $8m$ universe elements, $7m$ rows and $20m$ incidences. Labels require $O(\log(m + 1))$ bits. The parameter $K = q + 4m + k$ requires $O(\log(k + m + 1))$ bits. Sorting active labels and emitting the rows takes polynomial time in the explicit graph input length; the encoded output length is polynomial too. These bounds include edgeless graphs and arbitrarily long integer parameters. The CLI disables Python’s default decimal integer digit cap before parsing and printing; those conversions remain polynomial in encoded length.

The decoder scans the supplied basis against reconstructed rows, normalizes vertex pairs, and scans the edges. Its runtime is polynomial in combined source and target-output length, and its output uses at most $2m$ vertex labels of $O(\log(n + 1))$ bits. Neither map calls a Vertex Cover or Set Basis solver. The target-solving step remains NP-hard.

6. Conclusion

The five-set gadget yields a complete search reduction for the stated output semantics. The local count in Equation 2 makes recovery from an arbitrary ordinary basis explicit, including slack and redundant sets. This is a reproducible reconstruction of known hardness machinery, with no claim of a new complexity classification. Historical comparison with the unavailable original Stockmeyer and Jiang–Ravikumar proofs remains incomplete.

Bibliography

- [1] L. J. Stockmeyer, “The Set Basis Problem Is NP-Complete,” technical report, 1975. [Online]. Available: https://books.google.com/books/about/The_Set_Basis_Problem_is_Np_complete.html?id=NtDmtgAACAAJ
- [2] T. Jiang and B. Ravikumar, “Minimal NFA Problems Are Hard,” *SIAM Journal on Computing*, vol. 22, no. 6, pp. 1117–1141, 1993, doi: 10.1137/0222067.
- [3] H. Björklund and W. Martens, “The Tractability Frontier for NFA Minimization,” in *Automata, Languages and Programming, ICALP 2008, Part II*, in Lecture Notes in Computer Science, vol. 5126. Springer, 2008, pp. 27–38. doi: 10.1007/978-3-540-70583-3_3.

A. Verification and reproducibility

The fixed corpus contains 112 distinct legal source cases: 12 hand-designed edge cases and 100 seeded random graphs with at most six vertices. It has 80 YES and 32 NO instances. An independent SCIP 10.1.0 model solved the constructed Set Basis instances, checked their unions, and passed every answer through a fresh decoder process. The run covered 112 primary target outputs and one alternate valid basis. A separate Z3 4.16.0 verifier tested seven additional cases and nine outputs, including two alternate witnesses and three no-solution answers. It covered a two-edge composition and a graph with 100 labels but one edge. A 4301-digit parameter regression and an independent long-label review check passed after the CLI repair. These finite checks support the implementation, not the universal theorem.

The repository uses Python 3.12.14 and uv 0.12.17. The prepared oracle requires SCIP 10.1.0 on the command path; the separate verifier uses locked Z3 4.16.0. From the repository root, reconstruct the environment and run the checks:

```
uv sync --locked
uv run --locked python campaigns/vertex-cover-set-basis/work/test_algorithm.py
uv run --locked python campaigns/vertex-cover-set-basis/work/test_check.py
uv run --locked python campaigns/vertex-cover-set-basis/work/check.py --self-test
uv run --locked python campaigns/vertex-cover-set-basis/work/check.py --candidate
campaigns/vertex-cover-set-basis/work/algorithm.py
uv run --locked python campaigns/vertex-cover-set-basis/work/verify.py --candidate
campaigns/vertex-cover-set-basis/work/algorithm.py
```

For the two algorithm modes directly, run `uv run --locked python campaigns/vertex-cover-set-basis/work/algorithm.py` with a source JSON object on standard input. Add `--extract` and supply a JSON object with keys `source` and `target_solution` to recover an output. The exact JSON schemas and membership predicates are in `contract.md`. The corpus generator and fixed seeds are in `check.py` and `cases.json`. Typst 0.15.1 compiles this manuscript with `typst compile manuscript.typ manuscript.pdf` from the `work/` directory.